Building and Operating Agentic AI Systems with One or More Agents

Whitepaper + Standard Operating Procedure

Prepared with AI by Garrett Spear

Version 1.1 | Generated: April 25, 2026

1. Document Meta

Document owner	AI / Platform Engineering, Security Architecture, or designated Enterprise AI Product Owner
Application	Agentic AI systems that plan, reason, call tools, operate software workflows, create code, modify infrastructure, or interact with enterprise data
Suggested use	Internal whitepaper and baseline SOP package, ready for tailoring to a specific organization, project, and compliance regime.
Required review cadence	At kickoff, before pilot, before production, after material incident, and at least quarterly
Self-assessment result	99.1 / 100 against the baseline criteria summarized in Appendix D; see Appendix E for the score sheet summary.

2. Purpose

This whitepaper defines an enterprise-ready approach for designing, governing, implementing, validating, and operating agentic AI systems that use one or more agents. It explains why agentic AI systems require more than prompt engineering or an AGENTS.md file alone: they require explicit architecture, repository-based control artifacts, agent manifests, reusable skills, tool contracts, policy gates, evaluation harnesses, auditability, runtime controls, and operating procedures.

The purpose of this document is to provide a practical reference model for teams that need to move from experimental agent use toward governed enterprise implementation. It describes the core design principles, required architecture files, repository structure, autonomy levels, security controls, quality gates, implementation patterns, and operational expectations needed to make agentic systems explicit, versioned, testable, auditable, and safe to operate.

The detailed Standard Operating Procedure is included as Appendix A. The SOP converts the whitepaper's architecture and governance model into a repeatable step-by-step process for project intake, risk classification, architecture selection, agent decomposition, repository initialization, instruction design, skill design, tool governance, memory design, security controls, evaluation, deployment, ITSM integration, and continuous improvement.

3. Scope

This document applies to enterprise agentic AI systems in which one or more AI agents can observe context, reason over multi-step objectives, use tools, coordinate with humans or other agents, or affect business, software, security, infrastructure, operational, or governance workflows.

The whitepaper is intended for architecture, platform engineering, security engineering, DevOps, SRE, SOC, ITSM, compliance, product, and business-process teams that are evaluating or implementing agentic AI capabilities. It is especially relevant when an AI-enabled system may do any of the following:

- Observe enterprise data, tickets, logs, code, cloud resources, repositories, documents, user requests, or operational telemetry.
- Maintain context or memory across a task, session, project, incident, or workflow.
- Plan multi-step work rather than simply answering a single isolated prompt.

- Use tools, APIs, command-line utilities, MCP servers, CI/CD pipelines, workflow systems, ITSM platforms, cloud services, or infrastructure automation.
- Draft, create, review, approve, deploy, remediate, roll back, or document changes to code, infrastructure, configuration, runbooks, tickets, change records, evidence packages, or operational records.
- Coordinate work among multiple agents, specialized tools, human approvers, external systems, or downstream automation.
- Operate at any autonomy level from answer-only advisory support through narrowly scoped autonomous remediation.

This document covers internal development agents, DevOps agents, security and SOC agents, SRE agents, ITSM and change-management agents, compliance evidence agents, document-analysis agents, business-process agents, and orchestrator-led multi-agent systems.

This document does not treat a standalone chatbot, static prompt, or markdown instruction file as a complete agentic AI system. A system becomes agentic, for the purposes of this whitepaper, when it combines model reasoning with context, memory, planning, tool use, workflow execution, or operational decision support under defined governance controls.

4. Key Definitions

Term	Definition
Agent	A model plus a harness: instructions, state, memory, tools, permissions, feedback loops, guardrails, and observability. A markdown file alone is not an agent.
Harness	The runtime and control layer around the model that manages identity, tools, memory, state, approvals, retries, logging, policy checks, and handoffs.
AGENTS.md	A repository-level or directory-level instruction file for coding or operational agents. It should be concise, imperative, and version controlled.
SKILL.md	The required entry point for a reusable skill package. A skill is a focused procedure, often with scripts, references, examples, and assets that an agent can load when a task matches the skill description.
Agent manifest	A versioned file such as agents/<agent-name>/agent.yaml that defines the agent's purpose, tools, permissions, model defaults, escalation path, and approval boundaries.
Tool contract	A machine-readable specification for an API, command, MCP server, Lambda, script, or workflow that an agent may invoke, including inputs, outputs, permissions, rate limits, and failure behavior.
Autonomy level	The permitted action class for an agent, ranging from answer-only to read-only, draft-only, non-production write, production write with approval, and narrowly scoped autonomous remediation.
Black box recorder	Tamper-evident event logging of prompts, context sources, model decisions, tool calls, policy checks, approvals, outputs, exceptions, and human overrides.

5. Design Principles

This whitepaper adopts a C-P-A operating model — Context, Planning, and Action — informed by modern agentic DevOps architecture patterns. In this model, context includes perception and memory; planning includes reasoning, reflection, and policy evaluation; and action includes controlled tool use, verification, feedback, and audit logging.

In an agentic architecture, the AI model layer, usually an LLM, is primarily used for interpretation, planning, reasoning, decision support, and response generation. Supporting components may provide additional context through retrieval, analytics, rules, telemetry, memory stores, or indexed knowledge sources. Retrieval-augmented generation (RAG) may be used—where embedding models and vector indexes find relevant context—before a large language model (LLM) follows a reason and act (ReAct) workflow to generate a plan, response, or tool-use request. Orchestration layers coordinate iterative reason-and-act workflows that deconstruct an input, event, or objective into stepwise investigative and decision cycles. System integrations provide governed interfaces to approved tools, APIs, workflow platforms, infrastructure systems, and enterprise services beyond the immediate model runtime.

Appendix C, Figure 2 provides an example sequence diagram showing how the C-P-A model can appear in an operational SRE-style agent workflow: perception captures a user command and system context, planning/reasoning selects investigative and corrective steps, and action occurs through governed tool calls with state persistence and feedback.

- Repository-as-control-plane: the repo must contain the stable contract between humans, agents, tools, and deployment automation.
- AGENTS.md as a map, not an encyclopedia: keep the root file concise and point to deeper docs, skills, policies, and runbooks.
- Skills for repeatable procedures: package recurring work as focused SKILL.md-based skill directories rather than one-off prompts.
- Least privilege by default: no agent receives write permissions until a risk tier, business owner, tool owner, and approval path are documented.
- Deterministic gates around probabilistic reasoning: models may reason probabilistically, but execution must pass deterministic schemas, policy checks, tests, and approvals.
- Human-on-the-loop before human-out-of-the-loop: production autonomy must be earned with metrics, eval history, scoped blast radius, and rollback controls.
- Traceability from intent to action: every agent objective must trace to requirements, tool permissions, evals, logs, and change records.
- No hidden context: prompts, instructions, skills, and critical assumptions belong in version control, not in chat history.
- One source of truth with compatibility bridges: use AGENTS.md and skill packages as canonical portable assets; bridge to tool-specific files only when necessary.

Implementation pattern:

Practical implementations of the C-P-A model may use prompt chaining, routing, parallelization, reflection/producer-critic review, tool use, goal setting and monitoring, resource-aware optimization, and evaluation/monitoring. Appendix F provides MIT-licensed or license-audited prototype code examples for these mechanics. Those examples are illustrative only and do not replace the architecture, manifest, policy, evaluation, approval, observability, ITSM, and runtime controls required by this SOP.

6. Standard Repository Blueprint

The following structure is the recommended enterprise baseline. Remove unused folders only after documenting why they are not applicable.

```
repo-root/
  README.md
  AGENTS.md
  CONTRIBUTING.md
  SECURITY.md
  CODEOWNERS
  CHANGELOG.md
  LICENSE
  .gitignore
  .editorconfig
  .env.example
  PLANS.md
  IMPLEMENT.md
  ACCEPTANCE.md
  .codex/
```

```
  config.toml
  profiles/
    local-dev.toml
    read-only-review.toml
    ci-agent.toml
.github/
  copilot-instructions.md          # bridge only; AGENTS.md remains canonical
  workflows/
    ci.yml
    evals.yml
    policy-checks.yml
  pull_request_template.md
agents/
  README.md
  orchestrator/
    agent.yaml
    instructions.md
    tools.allowlist.yaml
    permissions.yaml
    eval-profile.yaml
    escalation.md
  specialist-devops/
    agent.yaml
    instructions.md
  specialist-security/
    agent.yaml
    instructions.md
  specialist-change-manager/
    agent.yaml
    instructions.md
skills/
  README.md
  code-change-verification/
    SKILL.md
    scripts/
    reference/
    examples/
    tests/
  terraform-plan-review/
    SKILL.md
    scripts/
    reference/
    examples/
    tests/
  incident-triage/
    SKILL.md
    scripts/
    reference/
    examples/
    tests/
  change-record-management/
    SKILL.md
```

```
    scripts/
    reference/
    examples/
    tests/
prompts/
  README.md
  task-prompts/
    create-plan.prompt.md
    review-diff.prompt.md
    generate-runbook.prompt.md
  critic-prompts/
    security-critic.prompt.md
    architecture-critic.prompt.md
    policy-critic.prompt.md
instructions/
  README.md
  coding-standards.md
  logging-standards.md
  secure-coding.md
  data-handling.md
  human-approval-rules.md
  model-use-policy.md
docs/
  reference/
    system-overview.md
    operating-model.md
    business-requirements.md
    nonfunctional-requirements.md
    glossary.md
  architecture/
    c4-context.md
    c4-container.md
    c4-component.md
    sequence-flows.md
    trust-boundaries.md
    data-flow-diagram.md
    deployment-view.md
  adr/
    ADR-0001-agent-pattern.md
    ADR-0002-model-provider.md
    ADR-0003-memory-strategy.md
    ADR-0004-tool-protocol.md
    ADR-0005-autonomy-level.md
    ADR-0006-agent-consensus-and-arbitration.md
    ADR-0007-runtime-credential-pattern.md
  runbooks/
    deploy.md
    rollback.md
    incident-response.md
    eval-failure.md
    key-rotation.md
    model-degradation.md
```

```
    adversarial-incident-playbook.md
    credential-rotation.md
  threat-model/
    threat-model.md
    misuse-cases.md
    controls-matrix.md
    red-team-plan.md
schemas/
  agent-event.schema.json
  tool-call.schema.json
  plan.schema.json
  change-record.schema.json
  memory-record.schema.json
  evaluation-result.schema.json
  handoff.schema.json
  consensus.schema.json
tools/
  README.md
  registry.yaml
  registry.high-risk-example.yaml
  mcp/
    servers.yaml
    authz-policy.yaml
  aws/
    lambda-tools.yaml
    iam-boundaries.yaml
    credential-issuance-flow.yaml
    sts-attestation-policy.yaml
  itsm/
    change-record-api.yaml
    ticket-api.yaml
policies/
  README.md
  opa/
    agent_action_authorization.rego
    terraform_guardrails.rego
    data_access.rego
    tool_use.rego
    consensus_arbitration.rego
    runtime_identity.rego
  policy-tests/
    allowed_cases.json
    denied_cases.json
memory/
  README.md
  memory-policy.md
  retention-schedule.md
  reembedding-policy.md
  vector-confidence-decay.md
  purge-and-rollback.md
  source-registry.yaml
  indexes/
```

```
    README.md                        # do not commit generated vector stores unless approved
evals/
  README.md
  datasets/
  fixtures/
  graders/
  rubrics/
  red-team/
    telemetry-poisoning/
    memory-poisoning/
    agent-suspension-rollback/
  reports/
src/
  agent_runtime/
  orchestration/
  tools/
  memory/
  policy/
  observability/
tests/
  unit/
  integration/
  security/
  eval/
infra/
  terraform/
  cloudformation/
  docker/
scripts/
  bootstrap.sh
  verify.sh
  run_evals.sh
  run_policy_checks.sh
  generate_sbom.sh
observability/
  dashboards/
  alerts/
  log-schema.md
  slo-governance.md
  black-box-retention.md
audit/
  README.md
  evidence-index.md
```

7. Required Agentic Architecture Files

The following files are mandatory unless the project is formally exempted. Each file must have a named owner and acceptance criteria.

Artifact	Purpose	Minimum contents
README.md	Human-facing project introduction.	Purpose, quick start, high-level architecture, support path, human contribution flow.

Artifact	Purpose	Minimum contents
AGENTS.md	Canonical agent-facing repo instructions.	Repo map, setup/build/test/lint commands, done definition, prohibited patterns, security rules, escalation rules, links to deeper docs.
Nested AGENTS.md	Scoped module or service instructions.	Local invariants, module-specific commands, public API boundaries, anti-patterns, test requirements.
PLANS.md	Standard execution-plan template.	Problem statement, assumptions, files to inspect, tasks, risks, rollback, validation plan.
IMPLEMENT.md	Agent implementation workflow.	Step-by-step operating procedure for turning a plan into a change set.
ACCEPTANCE.md	Definition of done.	Functional, security, policy, test, documentation, and operational acceptance gates.
skills/<name>/SKILL.md	Reusable procedure package for a specific task.	Name, description/trigger, inputs, outputs, steps, success criteria, failure handling, scripts and references.
agents/<name>/agent.yaml	Runtime-readable agent manifest.	Role, model/profile, tools, permissions, memory access, autonomy level, owner, escalation, eval profile.
agents/<name>/instructions.md	Agent-specific behavior instructions.	Mission, boundaries, reasoning style, tool sequencing, handoff protocol, refusal/escalation rules.
tools/registry.yaml	Canonical registry of tools the agent may call.	Tool name, owner, endpoint/command, schema, auth, rate limits, risk tier, approval requirement.
schemas/*.schema.json	Contracts for plans, tool calls, events, memory, outputs, and handoffs.	JSON Schema with required fields, examples, validation command, version.
policies/opa/*.rego	Deterministic policy gates.	Allow/deny rules for tool use, data access, infra changes, production actions, and approvals.
evals/*	Quality, safety, and regression evaluation harness.	Datasets, fixtures, rubrics, graders, red-team cases, success thresholds, reports.
docs/architecture/*	System design record.	C4 views, trust boundaries, sequence flows, deployment, data flow, architecture decisions.
docs/threat-model/*	Agentic threat model.	Misuse cases, OWASP LLM/agentic mappings, controls, mitigations, residual risks.
docs/runbooks/*	Operations playbooks.	Deploy, rollback, incident response, evaluation failure, key rotation, model outage, tool outage.
observability/*	Monitoring and audit evidence.	Dashboards, alert rules, log schema, black-box recorder fields, evidence index.
.codex/config.toml	Codex-specific defaults when Codex is used.	Model profile, sandbox mode, approval policy, working directories, MCP config references.
.github/copilot-instructions.md	Compatibility bridge for GitHub Copilot environments.	Pointer to canonical AGENTS.md and any Copilot-specific usage constraints; avoid divergent rules.
docs/runbooks/adversarial-incident-playbook.md	Playbook for contested or manipulated operating environments.	Telemetry-integrity decision tree, suspension criteria, forensic snapshot procedure, rollback path, and recovery approval rules.
docs/runbooks/credential-rotation.md	Runbook for runtime credential issuance and rotation.	Short-lived token issuance flow, attestation evidence, approval linkage, rotation cadence, revocation process, and audit references.
schemas/consensus.schema.json	Contract for multi-agent arbitration and conflict handling.	Specialist outputs, confidence, risk score, business-owner weighting, combined risk calculation, tie-breaking result, and human-arbitration requirement.
policies/opa/consensus_arbitration.rego	Policy gate for conflicting agent recommendations.	Allow/deny/escalate rules, weighted-priority evaluation, combined risk threshold, duplicate-remediation prevention, and human-arbitration triggers.
memory/reembedding-policy.md	Lifecycle policy for retrieval indexes and vector stores.	Re-embedding cadence, source authority tiers, confidence decay, deprecation rules, purge triggers, rollback validation, and owner review cadence.
observability/slo-governance.md	Operational SLO and automated governance specification.	SLIs/SLOs, breach thresholds, autonomy-reduction rules, approval escalation, post-mortem ticket creation, and recovery criteria.
observability/black-box-retention.md	Retention and access-control rules for audit evidence.	Retention period, access roles, legal hold rules, PII redaction requirements, evidence immutability, and review process.
tools/registry.high-risk-example.yaml	Example high-risk tool contract.	Inputs, outputs, idempotency key, error codes, approval requirement, timeout behavior, rollback reference, and required audit fields.

Implementation examples:

Code examples, scripts, or sample harnesses may be stored in the relevant skill examples/, scripts/, tests/, or evals/ directories when they are traceable to a skill, manifest, tool contract, schema, policy gate, or evaluation profile. Third-party code may be included only when the applicable license is verified, attribution is preserved, and production-hardening requirements are documented. Appendix F provides the standard pattern for adding these examples without weakening the control-plane model.

8. Standard AGENTS.md Template

Use this root AGENTS.md as a starter. Keep it short and link outward.

```
# AGENTS.md

## Operating Mode
- Treat this repository as the source of truth.
- Plan before changing code, infrastructure, prompts, policies, or agent manifests.
- Do not rely on chat history for requirements. If requirements are missing, document assumptions in the plan.

## Repository Map
- `docs/reference/` contains business and system context.
- `docs/architecture/` contains C4, data flow, sequence, and trust-boundary views.
- `agents/` contains agent manifests and agent-specific instructions.
- `skills/` contains reusable procedures. Load the relevant `SKILL.md` before performing matching work.
- `tools/` contains tool contracts and allowlists.
- `schemas/` contains validation contracts for plans, events, tool calls, memory, and outputs.
- `policies/` contains deterministic policy gates.
- `evals/` contains test fixtures, red-team cases, and scoring reports.
- `infra/` contains deployment automation.

## Required Workflow
1. Read this file and the nearest nested `AGENTS.md`.
2. Read the relevant `agents/<name>/agent.yaml` and `instructions.md` when acting as a named agent.
3. Read relevant `SKILL.md` files before using a skill.
4. Create or update a plan using `PLANS.md`.
5. Make the smallest safe change.
6. Run `scripts/verify.sh`.
7. Run relevant unit, integration, policy, security, and eval checks.
8. Update docs, schemas, prompts, or runbooks when behavior changes.
9. Summarize files changed, tests run, risks, and residual issues.

## Safety and Security Rules
- Never print, commit, or persist secrets.
- Never broaden IAM, network, data, or production permissions without policy and owner approval.
- Never call a tool that is not in `tools/registry.yaml`.
- Never bypass OPA/policy checks or approval gates.
- Never take production write action unless the agent manifest and change record authorize it.

## Definition of Done
- Tests pass.
- Policy checks pass.
- Evals pass or exceptions are documented and approved.
- Audit events are emitted for agent decisions and tool calls.
- Documentation and runbooks are updated.
- Residual risk is recorded.
```

9. Standard SKILL.md Template

A skill should be focused on one repeatable job. Use instructions alone unless deterministic scripts are needed.

```
---
name: terraform-plan-review
description: Use this skill whenever an agent creates, reviews, or modifies Terraform or an
infrastructure plan.
owner: Platform Engineering
risk_tier: high
required_inputs:
  - changed_files
  - terraform_plan
  - environment
outputs:
  - review_summary
  - risk_findings
  - policy_check_results
  - approval_recommendation
---

# Terraform Plan Review Skill

## Purpose
Review infrastructure changes for safety, cost, security, compliance, and rollback readiness.

## Trigger Conditions
Use this skill when a task includes Terraform, cloud infrastructure, IAM, networking, storage,
Kubernetes, production deployment, or cost-affecting resources.

## Procedure
1. Identify the target environment and blast radius.
2. Read applicable architecture docs, ADRs, IAM boundaries, and OPA policies.
3. Run formatting and validation commands.
4. Generate or inspect the Terraform plan.
5. Check for IAM broadening, public exposure, encryption changes, destructive actions, data retention
changes, and cost changes.
6. Run OPA policy checks.
7. Produce a risk summary and approval recommendation.
8. If production-impacting, require a change record and human approval before apply.

## Required Commands
- `terraform fmt -check -recursive`
- `terraform validate`
- `terraform plan -out=tfplan`
- `scripts/run_policy_checks.sh`

## Success Criteria
- No unapproved destructive changes.
- No unapproved privilege broadening.
- Required tags, logging, encryption, and monitoring are present.
- Rollback path is documented.
- Change record exists for production changes.

## Failure Handling
Stop and escalate if the plan includes unknown destructive actions, unscoped IAM permissions, public data
exposure, failed policy checks, missing rollback, or missing owner approval.
```

10. Agent Manifest Template

The manifest makes the runtime, scope, and approval boundaries explicit. It prevents the common mistake of treating a prompt file as if it were an actual governed agent.

```yaml
# agents/specialist-devops/agent.yaml
agent_id: specialist-devops
display_name: DevOps Automation Agent
owner: platform-engineering
business_owner: application-platform
runtime:
  harness: openai-agents-sdk-or-enterprise-runtime
  environment: dev
  default_model_profile: balanced
  max_steps: 12
  timeout_seconds: 900
autonomy:
  level: L3
  allowed_actions:
    - read_repository
    - propose_code_change
    - create_pull_request
    - modify_non_production_infrastructure_with_approval
  prohibited_actions:
    - production_apply_without_change_record
    - secret_access_unless_explicitly_scoped
    - direct_database_mutation
tools:
  allowlist_ref: agents/specialist-devops/tools.allowlist.yaml
  registry_ref: tools/registry.yaml
permissions:
  iam_boundary_ref: tools/aws/iam-boundaries.yaml
  data_access: non_sensitive_project_data
  network_access: restricted
memory:
  read_sources:
    - docs/reference
    - docs/architecture
    - runbooks
  write_memory: false
policy:
  opa_required: true
  policies:
    - policies/opa/tool_use.rego
    - policies/opa/terraform_guardrails.rego
approvals:
  required_for:
    - production_change
    - permission_broadening
    - destructive_action
    - external_data_transfer
observability:
```

```
    event_schema: schemas/agent-event.schema.json
    log_destination: centralized_audit_log
  evals:
    profile_ref: agents/specialist-devops/eval-profile.yaml
  escalation:
    human_queue: platform-oncall
    itsm_queue: change-management
```

11. Autonomy Levels and Approval Rules

Level	Permitted behavior	Typical use	Required controls
L0 - Answer only	No tools or enterprise data access.	Brainstorming and general education.	No secrets; no authoritative operational claims unless sourced.
L1 - Read-only	Can inspect approved docs, logs, tickets, code, or telemetry.	Analysis, summarization, triage.	Identity, data access logging, no mutation tools.
L2 - Draft only	Can draft code, tickets, PR descriptions, plans, or change records but cannot submit or execute.	Design, review, proposal generation.	Human approval before persistence or submission.
L3 - Non-production write	Can create branches, PRs, sandbox resources, or dev/test changes.	Coding agents, test environment automation.	Policy checks, CI tests, scoped credentials, rollback plan.
L4 - Production write with approval	Can execute approved production action after change record and human gate.	SRE, incident remediation, deployment assistant.	ITSM change, OPA allow decision, dual control for high-risk actions, black-box logging.
L5 - Narrow autonomous production remediation	Can execute pre-approved, low-blast-radius production runbooks without per-action human approval.	Restart known failed worker, scale within bounds, roll back failed canary.	Pre-approved runbook, hard limits, automatic rollback, alerting, post-action review, suspension switch.

12. Single-Agent and Multi-Agent Reference Architectures

Pattern	Use when	Enterprise guardrails
Single agent with tools	The task domain is narrow, tools are few, and risk is low to moderate.	One manifest, one tool allowlist, clear approval gates, focused skills.
Orchestrator + specialists	Work requires planning plus domain-specific execution such as DevOps, security review, and change management.	One orchestrator owns workflow state; specialists return structured outputs; no uncontrolled agent loops.
Supervisor / worker	A supervising agent delegates bounded subtasks to workers.	Supervisor verifies outputs, workers lack production permissions, all handoffs use schemas.
Event-driven pipeline	Work is triggered by events such as SIEM alerts, CI failures, cloud drift, or ticket updates.	Queue-based state, idempotent workers, replayable events, rate limits, policy gates.
Human-in-the-loop co-pilot	Human judgment is mandatory or trust is still being established.	Agent drafts; human approves; audit logs preserve suggestions and final decisions.
Autonomous runbook executor	The task is known, low-blast-radius, reversible, and repeatedly validated.	Pre-approved runbooks, hard limits, monitoring, kill switch, post-action review.
Delegated consensus with arbitration	Multiple specialists may produce conflicting structured recommendations, risk findings, or remediation plans.	Use schemas/consensus.schema.json and policy arbitration; require human review when combined risk exceeds threshold or no deterministic tie-breaker applies.

Implementation pattern:

Routing, parallelization, and producer-critic review may appear inside single-agent, orchestrator-plus-specialist, supervisor/worker, event-driven, human-in-the-loop, or delegated-consensus architectures. Parallel execution must not create duplicate authority over final action selection. The orchestrator or supervising control layer remains responsible for workflow state, result aggregation, conflict detection, policy checks, audit records, and escalation. See Appendix F for prototype examples and hardening notes.

13. Minimum Security Control Set

- Identity: unique service identity per agent and per environment.
- Authorization: tool allowlists, scoped IAM, network boundaries, and policy-as-code decisions.

- Input safety: treat external content as untrusted; isolate retrieved data from instructions; add indirect prompt injection tests.
- Output safety: validate all outputs before execution; never feed untrusted model output directly to shell, SQL, Terraform apply, or admin APIs.
- Data protection: classify data, restrict retrieval, redact secrets, enforce retention, and prevent unauthorized export.
- Supply chain: pin dependencies, scan containers and packages, verify MCP/tool servers, and maintain SBOMs.
- Auditability: log complete decision/action chains, policy checks, approvals, and evidence links.
- Resilience: timeouts, retries, circuit breakers, cost controls, model fallback, tool outage handling, and kill switches.
- Human control: approval gates, escalation paths, emergency disablement, and post-incident review.

Adversarial incident readiness: write-capable agents must be suspended when telemetry integrity, retrieval context, prompts, or tool outputs are suspected of adversarial manipulation. The suspension process must preserve forensic snapshots of session state, retrieved context identifiers, memory indexes, vector-store metadata, policy decisions, and tool-call evidence before purge or rollback.

Runtime identity and credential lifecycle: production-capable agents must use short-lived, scoped credentials issued just in time through an approved identity flow. Credential issuance must be attributable to an agent identity, approved purpose, environment, change record, and policy decision. Long-lived shared credentials must not be used for production-capable agents.

Multi-agent conflict resolution: orchestrator-led and delegated-consensus patterns must define how conflicting specialist outputs are detected, weighted, resolved, or escalated. A human arbitrator is required when combined risk exceeds a documented threshold or when policy cannot deterministically select a safe path.

Memory lifecycle and vector-store drift: memory stores and retrieval indexes must define retention windows, source authority tiers, re-embedding cadence, confidence decay, deprecation rules, poisoning response, purge triggers, and rollback validation for removed or corrected memory.

Operational SLO governance: production pilots and production systems must define SLIs and SLOs for success rate, escalation rate, policy denial rate, tool failure rate, unsafe-output prevention, latency, cost, and human override rate. SLO breaches must trigger documented governance responses, such as reducing autonomy, requiring human approval for the next defined number of actions, opening a post-mortem ticket, or suspending a tool.

Black-box recorder data handling: black-box recorder retention, access controls, legal-hold expectations, PII redaction, and evidence immutability must be documented before production use.

Example-code boundary: Appendix F examples demonstrate implementation mechanics only. Example code must not be interpreted as production authorization. Any prompt chain, router, parallel worker, critic loop, tool call, resource optimizer, goal monitor, or evaluation harness must be wrapped with identity, authorization, input validation, output validation, rate limits, policy enforcement, logging, approval gates, and black-box recorder events before operational use.

14. Quality Gates

Gate	Must pass before
Architecture review	Any implementation work beyond prototype
Threat model review	Tool access to enterprise systems or sensitive data
Agent manifest review	Any runtime deployment
Tool contract review	Tool registration or MCP server exposure
Policy-as-code review	Any write or production-impacting action
Eval threshold	Pilot and production promotion
Security test / red team	Production or high-risk pilot
ITSM approval	Production change

Gate	Must pass before
Observability readiness	Any sustained runtime deployment
Operational handoff	Production ownership transfer
Adversarial incident playbook review	Write-capable pilot or production deployment in contested or security-sensitive environments
Runtime credential lifecycle review	Production-capable tool access or L4/L5 autonomy
Consensus and arbitration review	Multi-agent workflows with conflicting or weighted specialist outputs
Memory lifecycle review	Persistent memory, vector indexes, or long-term retrieval sources
Operational SLO governance review	Pilot promotion, production promotion, and any autonomy-level increase

15. Acceptance Checklist

- ☐ Business owner and technical owner assigned.
- ☐ Use case, data classification, risk tier, and autonomy level documented.
- ☐ C4 context/container views and trust boundaries completed.
- ☐ Root AGENTS.md created and reviewed.
- ☐ Nested AGENTS.md files created where local rules differ.
- ☐ Agent manifests and instructions created for all agents.
- ☐ Skills created for repeatable workflows and tested for correct triggering.
- ☐ Tool registry, tool contracts, and MCP/API configuration completed.
- ☐ Schemas created for plans, tool calls, outputs, memory, and audit events.
- ☐ OPA or equivalent policy checks implemented for high-risk actions.
- ☐ Evals, regression tests, and red-team cases created.
- ☐ CI/CD runs tests, schema validation, policy checks, and evals.
- ☐ ITSM/change-record workflow implemented for production-impacting actions.
- ☐ Observability and black-box recorder deployed.
- ☐ Rollback, kill switch, incident response, and decommission procedures documented.
- ☐ Adversarial incident playbook completed and tested against telemetry manipulation, prompt injection, and suspension/rollback scenarios.
- ☐ Production-capable agents use short-lived, scoped credentials with documented issuance, attestation, rotation, revocation, and audit linkage.
- ☐ Multi-agent conflict resolution schema and arbitration policy completed where specialists can return conflicting plans.
- ☐ Memory lifecycle policy includes retention windows, re-embedding cadence, confidence decay, deprecation rules, purge triggers, and rollback validation.
- ☐ Operational SLOs and automated governance triggers are defined for pilot and production stages.
- ☐ Black-box recorder retention, access control, PII redaction, and evidence immutability rules are documented.

16. Practical Implementation Sequence

1. Create the repo skeleton and document the business use case.
2. Write the root AGENTS.md as the concise operating map.
3. Create architecture docs, ADRs, and threat model.
4. Define agents and write their manifests.
5. Define skills for repeatable workflows.
6. Register tools and write tool contracts.
7. Write schemas and policy-as-code gates.
8. Implement the runtime harness and tool wrappers.
9. Build the eval suite and red-team cases.

10. Create the adversarial incident playbook, runtime credential lifecycle runbook, consensus schema, memory lifecycle policy, and SLO governance triggers.
11. Implement short-lived credential issuance, policy arbitration, memory purge/rollback validation, and SLO-based autonomy reduction controls.
12. Deploy a read-only pilot.
13. Promote to draft-only and non-production write after passing evals.
14. Promote to production write with approval after security, change-management, and operational readiness reviews.
15. Consider narrowly scoped autonomy only after repeated safe performance and documented owner approval.

17. Change-Record Schema Summary

For production-affecting agentic actions, the change record must capture the following minimum fields:

Field group	Required fields
Identity	change_id, requester, agent_id, human_approver, business_owner, technical_owner
Scope	service, environment, affected_assets, data_classification, autonomy_level, risk_tier
Intent	business_reason, desired_outcome, user_request, agent_plan_ref
Action	tool_calls, commands, infrastructure_plan_ref, code_diff_ref, deployment_ref
Controls	policy_decision_ids, eval_report_ref, test_results_ref, security_review_ref
Timing	requested_at, approval_at, execution_window, completed_at
Rollback	rollback_plan, rollback_owner, rollback_validation, backout_deadline
Evidence	black_box_event_ids, logs_ref, screenshots_ref, monitoring_ref, post_implementation_review
Runtime identity	credential_issuance_id, credential_ttl, attestation_ref, revocation_ref, runtime_identity_policy_ref
Governance response	slo_breach_id, autonomy_reduction_action, suspension_decision, arbitration_ref, post_mortem_ticket_ref

18. Final Self-Assessment Note

The SOP was assessed against the separate baseline criteria document and iteratively adjusted until the self-score exceeded 99%. The final score is not a certification; it is an internal quality assertion based on measurable coverage, specificity, executability, traceability, security, and enterprise readiness. The remaining gap to 100% is that some organization-specific details, such as exact cloud accounts, enterprise risk appetite, compliance mappings, approved model providers, and specific ITSM system fields, must be tailored by the adopting organization.

The baseline criteria used to assess this SOP are provided in Appendix D, and the completed score sheet is provided in Appendix E.

19. Source Notes

The procedure uses the following public sources as design inputs. These are not the only valid sources for enterprise adoption; they are the baseline references used for this document.

ID	Publisher	Source	URL
S1	Harness	The Agent-Native Repo: Why AGENTS.MD is the New Standard	https://www.harness.io/blog/the-agent-native-repo-why-agents-md-is-the-new-standard
S2	OpenAI Developers	Custom instructions with AGENTS.md - Codex	https://developers.openai.com/codex/guides/agents-md
S3	OpenAI Developers	Best practices - Codex	https://developers.openai.com/codex/learn/best-practices
S4	OpenAI Developers	Agent Skills - Codex	https://developers.openai.com/codex/skills
S5	OpenAI	Harness engineering: leveraging Codex in an agent-first world	https://openai.com/index/harness-engineering/
S6	NIST	AI Risk Management Framework	https://www.nist.gov/itl/ai-risk-management-framework
S7	NIST AI Resource Center	AI RMF Core: govern, map, measure, manage	https://airc.nist.gov/airmf-resources/airmf/5-sec-core/
S8	OWASP	Top 10 for Large Language Model Applications	https://owasp.org/www-project-top-10-for-large-language-model-applications/
S9	OWASP GenAI Security Project	Top 10 for Agentic Applications for 2026	https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
S10	Model Context Protocol	MCP Specification	https://modelcontextprotocol.io/specification/2025-06-18
S11	Open Policy Agent	OPA documentation	https://www.openpolicyagent.org/docs

ID	Publisher	Source	URL
S12	Open Policy Agent	Using OPA in CI/CD Pipelines	https://www.openpolicyagent.org/docs/cicd
S13	Chris Reddington	AGENTS.md and SKILL.md: building a reusable agent toolbox	https://chrisreddington.com/blog/building-your-agent-toolbox/
S14	Antonio Gulli	Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems; selected MIT-licensed code examples and pattern concepts used as appendix implementation references	Preprint/book source and cited code examples; include only verified MIT-licensed code with attribution and license notice retained.

20. Appendices

20.1 Appendix A: SOP Procedure

20.1.1 Intake and Use-Case Classification

1. Create a use-case record that states the business objective, owner, users, target systems, data involved, expected actions, value proposition, and risk tier.
2. Classify whether the system is advisory, workflow-assistive, tool-using, infrastructure-changing, security-impacting, or production-remediating.
3. Assign a preliminary autonomy level from L0-L5. Default to the lowest level that can deliver value.
4. Document explicit non-goals and forbidden outcomes.
5. Open an ADR if the use case introduces a new model provider, tool protocol, memory store, or production autonomy pattern.

20.1.2 Enterprise Risk Mapping

6. Map the use case to NIST AI RMF governance, mapping, measurement, and management activities.
7. Identify data sensitivity, regulatory drivers, system criticality, user population, failure modes, and human impact.
8. Create a misuse-case list that includes prompt injection, tool misuse, privilege abuse, memory poisoning, supply-chain compromise, unsafe code execution, model denial of service, data leakage, and excessive agency.
9. Decide whether the system is allowed to process regulated, confidential, customer, employee, source code, security telemetry, or production secrets data.
10. Record residual risk and required compensating controls.

20.1.3 Architecture Pattern Selection

11. Choose a single-agent, orchestrator-plus-specialists, supervisor-worker, event-driven pipeline, or human-in-the-loop workflow pattern.
12. Prefer one accountable orchestrator when multiple specialist agents are used. Avoid uncontrolled peer-to-peer agent loops.
13. If delegated consensus is used, document the arbitration model, risk threshold, tie-breaking mechanism, and human-arbitration conditions in ADR-0006-agent-consensus-and-arbitration.md and schemas/consensus.schema.json.
14. Define boundaries between reasoning, retrieval, tool execution, policy decision, approval, memory, and audit components.
15. Create C4 context and container views before implementation. Add component views for runtime services, agents, tools, and policy gates before production.
16. Document trust boundaries, data flows, identity handoffs, network zones, and deployment environments.
17. For a conceptual reference model showing the relationship between perception, memory, orchestration, planning, policy governance, tool execution, infrastructure action, and observability feedback, see Appendix B, Figure 1.

20.1.4 Agent Decomposition

18. Identify each agent by mission, not by vague persona. Example: Change Manager Agent, DevOps Implementation Agent, Security Reviewer Agent, Evidence Collector Agent.

19. For each agent, define inputs, outputs, tools, memory access, decision rights, escalation conditions, and maximum autonomy level.
20. Prevent duplicate authorities. Only one agent should own final action selection for a workflow unless a human approval system resolves conflicts.
21. Documented delegated-consensus patterns are allowed only when the orchestrator applies an explicit arbitration schema, logs the specialist outputs, calculates combined risk, and escalates to a human when risk or conflict thresholds are exceeded.
22. Create agent manifests under agents/<agent-name>/agent.yaml and specific instructions under agents/<agent-name>/instructions.md.
23. Add agent-to-agent handoff schema and logs if agents coordinate.

20.1.5 Repository Initialization

24. Create the standard repository blueprint or adapt an existing repo to the blueprint.
25. Add README.md for humans, AGENTS.md for agents, PLANS.md for planning, IMPLEMENT.md for execution, ACCEPTANCE.md for done criteria, and CODEOWNERS for governance.
26. Add nested AGENTS.md files only where local behavior differs from the root.
27. Create .codex/config.toml and tool-specific bridge files only if the organization uses those tools. The canonical content remains in AGENTS.md and skills.
28. Create an evidence index that maps requirements to files, tests, policies, evals, and approvals.

20.1.6 Instruction and Prompt Engineering

29. Write instructions imperatively: say what the agent must do, must not do, and how to verify completion.
30. Move reusable procedures into skills rather than repeating long prompts.
31. Move one-time task inputs into task prompts rather than bloating AGENTS.md.
32. Separate stable policy from dynamic task context. Stable instructions live in version control; dynamic user requests live in plans and work items.
33. Test instructions by running representative prompts and recording whether the agent loaded the right files, followed the right gates, and avoided known mistakes.

20.1.7 Skill Design

34. Create a skill only for repeatable work that benefits from packaged procedure, deterministic scripts, or reference materials.
35. Each skill must have a SKILL.md with trigger description, required inputs, steps, outputs, success criteria, failure handling, and owner.
36. Keep each skill focused on one job. Split broad skills into smaller ones such as code-change-verification, terraform-plan-review, incident-triage, and change-record-management.
37. Place deterministic utilities in scripts/, examples in examples/, reference material in reference/, and tests in tests/.
38. Add trigger tests to confirm the agent selects the skill when appropriate and does not select it when irrelevant.

20.1.8 Tool Contract and MCP/API Design

39. Register every callable tool in tools/registry.yaml before use.
40. Define inputs, outputs, authentication, authorization, rate limits, error semantics, timeout, idempotency, logging fields, and risk tier.
41. Use MCP servers or API gateways only through approved transport, identity, and authorization patterns.
42. Wrap destructive or high-risk tools in an approval and policy layer rather than exposing raw shell, cloud, database, or admin APIs directly.
43. Require schemas and contract tests for all tool calls.
44. High-risk tool contract example: A production restart tool in tools/registry.high-risk-example.yaml should define tool_id, owner, purpose, allowed_environment, required_approval, idempotency_key, input_schema, output_schema, timeout_seconds, retry_policy, rollback_reference, error_codes, audit_fields, and policy_refs.

45. Example fields: inputs include service_name, namespace, environment, change_id, reason, requested_by, and idempotency_key; outputs include status, affected_resources, started_at, completed_at, policy_decision_id, rollback_ref, and audit_event_id; error codes include POLICY_DENIED, APPROVAL_REQUIRED, INVALID_SCOPE, CONFLICTING_ACTION, TOOL_TIMEOUT, and ROLLBACK_REQUIRED.

20.1.9 Memory and Retrieval Design

46. Define what memory exists: session memory, project memory, long-term memory, vector retrieval, ticket history, runbooks, source docs, or audit logs.
47. Do not let agents write unreviewed long-term memory by default.
48. Classify memory sources by sensitivity, freshness, authority, and retention period.
49. Record source provenance in every retrieved context item.
50. Create a memory poisoning test set and purge process.
51. Define re-embedding cadence, confidence decay, source deprecation, and automated purge triggers for every persistent memory source or retrieval index.
52. Validate purge and rollback behavior in CI so corrected or removed source material cannot continue influencing agent planning through stale index entries.

20.1.10 Security and Governance Controls

53. Use least-privilege identities per agent and per tool, not a shared all-powerful service account.
54. Apply policy-as-code before write actions, privilege changes, production actions, external sends, and data export.
55. Require human approval for actions outside the agent's autonomy level.
56. Sanitize and validate model outputs before passing them to shells, databases, browsers, infrastructure tools, or downstream systems.
57. Log prompts, retrieved context identifiers, tool calls, policy decisions, approvals, outputs, and exceptions to the black-box recorder.
58. Add an adversarial incident playbook that defines telemetry-integrity checks, criteria for suspending write-capable agents, forensic snapshot requirements, rollback procedures, and recovery approvals.
59. For production-capable agents, require short-lived credentials, just-in-time issuance, runtime attestation, revocation paths, and audit linkage to the agent identity, policy decision, and change record.
60. Define black-box recorder retention periods, access controls, PII redaction rules, legal-hold expectations, and evidence immutability controls before production deployment.

20.1.11 Evaluation and Red-Team Program

61. Create eval datasets for normal tasks, edge cases, security cases, policy-denied cases, malformed input, tool failures, and regression bugs.
62. Use rubrics that score task success, instruction following, tool safety, security behavior, citation/provenance, cost, latency, and escalation correctness.
63. Run red-team tests for prompt injection, indirect prompt injection, goal hijack, unsafe tool chaining, privilege escalation, data exfiltration, and memory poisoning.
64. Add adversarial SecOps scenarios that simulate manipulated telemetry, poisoned retrieval context, malicious tool output, and active prompt injection inside operational feeds.
65. Validate that the agent suspends or reduces autonomy, preserves forensic evidence, avoids unsafe remediation, and triggers rollback or human escalation when adversarial conditions are detected.
66. Block production promotion until evals pass the documented threshold or exceptions are approved by the risk owner.
67. Store eval reports under evals/reports and link them from the release record.

Appendix F evaluation examples may be used as starter harness patterns only after adaptation to project-specific schemas, rubrics, red-team scenarios, policy-denied cases, release thresholds, and audit evidence requirements.

20.1.12 Build, Test, and CI/CD

68. All code, prompts, skills, agent manifests, tool contracts, schemas, and policies must be reviewed through pull requests.
69. CI must run unit tests, integration tests, schema validation, linting, policy checks, security scans, and relevant evals.
70. Infrastructure changes must produce a plan artifact and pass policy checks before apply.
71. Production deployment must be linked to a change record, release notes, rollback plan, and observability dashboard.
72. Generated artifacts must be reproducible from source.

20.1.13 ITSM and Change Management

73. Create or update a change record before any production-impacting action.
74. The change record must include requester, business service, affected systems, risk, rollback plan, validation plan, agent identity, tool actions, approval status, and evidence links.
75. Agents may draft change records at L2, submit for review at L3 if allowed, and execute only under L4/L5 rules.
76. Emergency changes still require after-action documentation and post-implementation review.
77. All change records must link to black-box recorder event IDs and deployment artifacts.

20.1.14 Deployment and Runtime Operations

78. Deploy agents with environment-specific identities, secrets, network restrictions, logging, and rate limits.
79. Separate development, test, staging, and production configurations.
80. Use feature flags or kill switches for high-risk tools and autonomous loops.
81. Monitor success rate, escalation rate, denied policy actions, tool failures, latency, cost, hallucination indicators, data access patterns, and user overrides.
82. Define operational SLIs and SLOs, such as agent-success-rate, escalation-rate per 1,000 actions, policy-denial-rate, tool-failure-rate, unsafe-output-prevention-rate, latency, cost, and human-override-rate.
83. When an SLO breach occurs, automatically reduce autonomy, require human approval for the next defined number of actions, suspend affected tools when appropriate, and open a post-mortem or governance-review ticket.
84. Define rollback, disablement, and incident-response procedures before enabling production use.

Resource-aware optimization examples in Appendix F may be used to control cost, latency, model choice, search escalation, and tool selection. Cost or latency optimization must never bypass policy, data-handling, security, approval, or audit requirements.

20.1.15 Continuous Improvement

85. Review incidents, agent mistakes, policy denials, failed evals, and human overrides weekly during pilot and at least monthly in production.
86. Update AGENTS.md only for stable recurring guidance; update skills for repeatable procedures; update prompts for task-specific entry points; update evals for every defect class.
87. Retire unused tools, stale skills, outdated instructions, and obsolete memory sources.
88. Run quarterly access reviews and instruction drift reviews.
89. Maintain a release history of agent behavior changes, model changes, tool changes, policy changes, and autonomy changes.

20.2 Appendix B: Reference Architecture for Agentic AI Systems

Figure 1 provides a conceptual reference model for an enterprise-grade agentic AI system. The diagram shows a governed control loop beginning with an operational trigger, such as an alert or ticket, flowing through a perception layer into a cognitive core that includes an LLM orchestrator, planning module, and self-reflection/critic step. It also illustrates

supporting memory systems, including the context window, vector database/retrieval layer, and internal documentation sources.

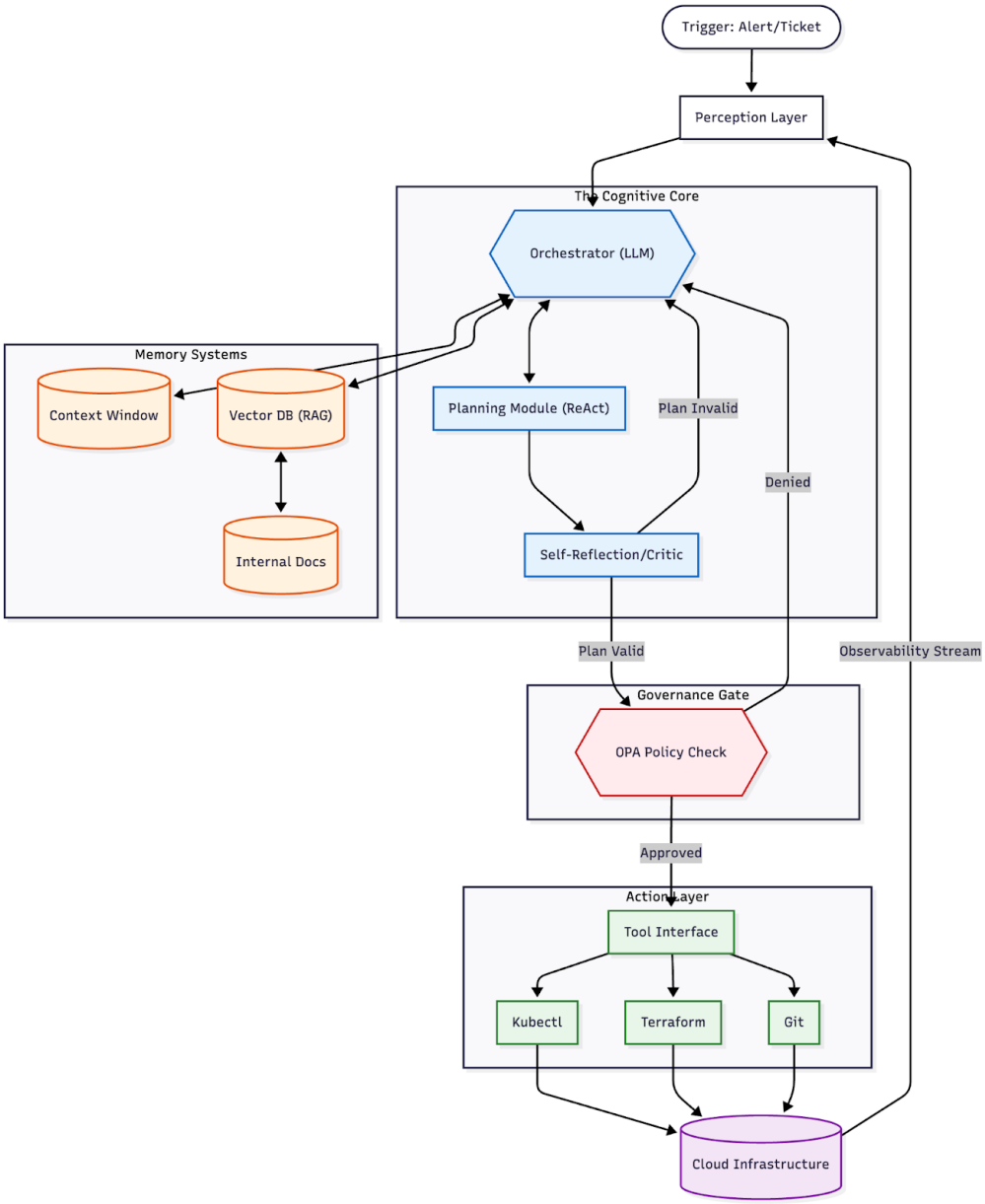
The model emphasizes that tool execution should not occur directly from model reasoning. Instead, plans should pass through a governance gate, such as an OPA policy check, before approved actions reach the action layer and invoke tools such as Kubernetes, Terraform, or Git. The observability stream closes the loop by feeding results, infrastructure state, and runtime telemetry back into the perception layer for continued monitoring, auditability, and improvement.

Recommended use:

This reference architecture should be used as a conceptual baseline when designing single-agent, orchestrator-led, or multi-agent enterprise systems. Implementation teams should adapt the model to the selected runtime, cloud environment, policy engine, ITSM platform, and approved tool registry, while preserving the core separation between perception, reasoning, memory, governance, action, and observability.

Figure 1. Agentic AI System Reference Architecture: Governed Trigger-to-Action Control Loop

Agentic AI in DevOps: Architecting autonomous infrastructure. (2026, February 11). Aditya Kashyap, Harness.io. https://www.harness.io/blog/agentic-ai-in-devops-the-architects-guide-to-autonomous-infrastructure



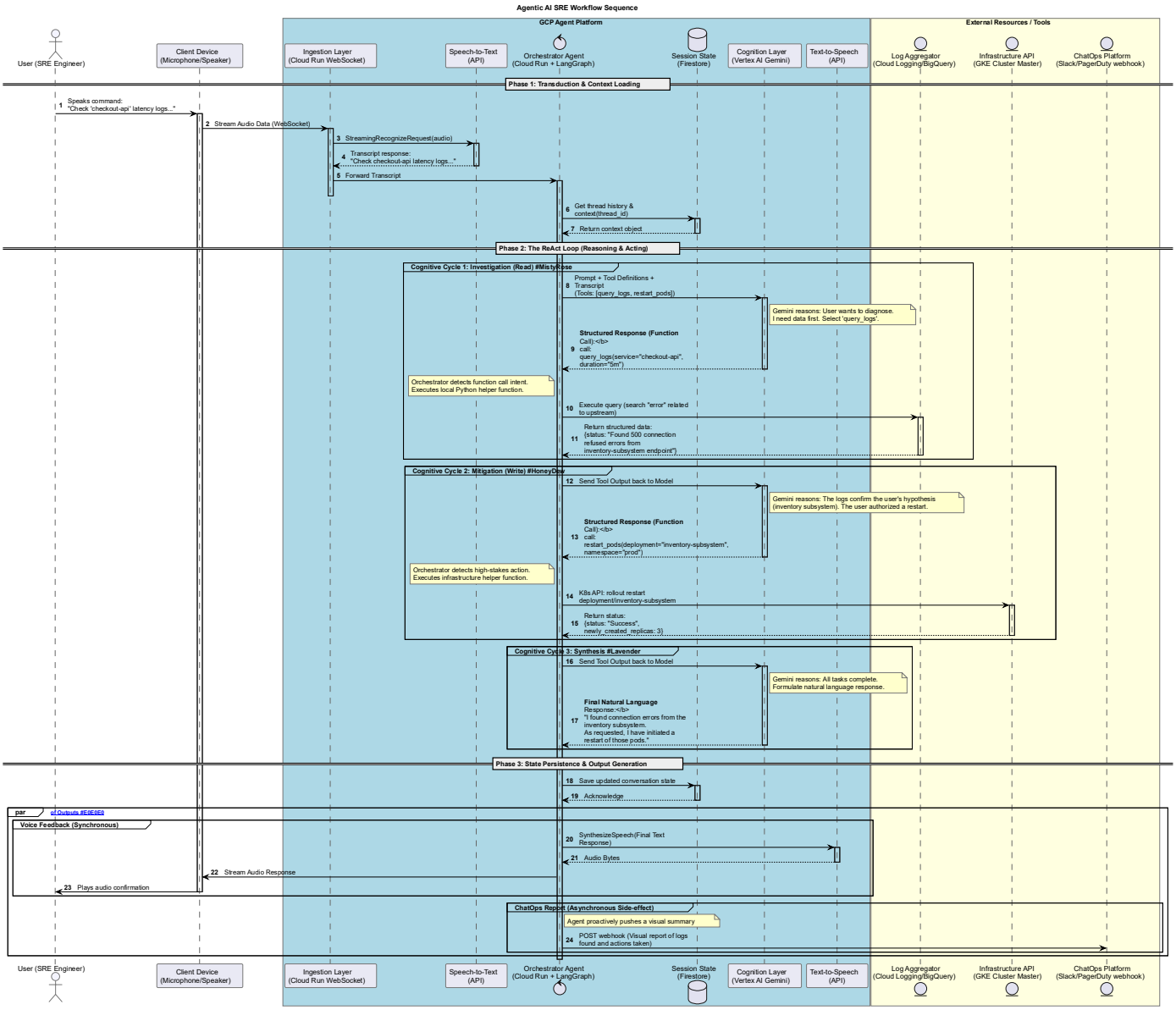
20.3 Appendix C: Example Agentic Workflow Sequence Diagram

Figure 2 provides an example sequence diagram for an agentic SRE workflow. The scenario begins with a user command, passes through a voice and ingestion layer, loads conversation context, invokes an orchestrator agent, reasons through investigative and corrective steps, calls external tools, persists state, and returns both a synchronous user response and an asynchronous ChatOps report.

The diagram is useful because it translates the conceptual reference architecture from Appendix B into a concrete workflow. It shows how perception, memory, reasoning, tool use, infrastructure action, state persistence, and feedback can be composed into a practical agentic operating loop.

In the example, the agent first investigates service latency by querying logs. After receiving structured evidence, the agent reasons about a mitigation step and invokes an infrastructure tool to restart a Kubernetes deployment. Finally, the agent synthesizes a human-readable response, saves the updated conversation state, produces voice feedback, and posts a summary to a ChatOps destination.

Figure 2. Example Agentic Workflow Sequence Diagram: Voice-Initiated SRE Investigation and Mitigation Loop



Implementation note:

This diagram should be treated as an illustrative workflow pattern, not a complete production control design. In an enterprise implementation, privileged or high-impact actions such as restarting pods, changing infrastructure, modifying deployments, disabling controls, or accessing sensitive data should pass through explicit governance controls. Those controls may include policy-as-code checks, human approval thresholds, ITSM change-record creation, dry-run validation, rollback planning, identity-aware authorization, and tamper-evident audit logging.

Recommended use:

Use this sequence diagram as an example of how the C-P-A model can be implemented in a real agentic workflow. Teams may adapt the pattern to other domains, such as security operations, incident response, cloud operations, ticket triage, compliance evidence collection, or infrastructure remediation, while preserving the separation between perception, reasoning, memory, governed tool use, action, and feedback.

20.4 Appendix D: SOP Assessment Baseline Criteria

The *Agentic AI SOP Assessment Baseline Criteria* defines the baseline criteria used to assess this SOP. These criteria are intended to make the SOP measurable, reviewable, and improvable rather than relying on subjective impressions of completeness. The criteria should be used during initial SOP approval, periodic review, architecture governance reviews, implementation readiness assessments, and post-implementation lessons-learned cycles. The assessment baseline evaluates whether the SOP provides sufficient detail for an enterprise team to design, build, test, secure, deploy, govern, and operate an agentic AI system using one or more agents.

Agentic_AI_SOP_Assessment_Baseline_C

Agentic_AI_SOP_Assessment_Baseline_Criteri

20.5 Appendix E: SOP Score Sheet

The *Agentic AI SOP Score Sheet* documents the completed assessment of this SOP against the baseline criteria in Appendix D. The purpose of the score sheet is to make the final quality assessment transparent, traceable, and repeatable.

Agentic_AI_SOP_Score_Sheet.docx

Agentic_AI_SOP_Score_Sheet.pdf

Final Score: **99 / 100**
Final Rating: **Excellent / Enterprise-ready**

The SOP meets the standard for an enterprise-ready agentic AI system development and implementation procedure. It provides a practical bridge between AI-agent repository conventions, secure enterprise architecture, tool-governed execution, policy-as-code, operational readiness, and measurable validation.

The SOP intentionally treats AGENTS.md as only one part of the agentic system control plane. It expands the implementation model to include skills, manifests, schemas, tool contracts, policies, evaluations, runbooks, CI/CD gates, change-management records, observability controls, and lifecycle governance.

The single point not awarded reflects the fact that no general-purpose SOP can be perfectly tailored to every enterprise without incorporating organization-specific tools, compliance obligations, approved cloud patterns, naming standards,

and operating procedures. Once those local details are added, the SOP can reasonably be treated as a 100-point internal implementation standard.

20.6 Appendix F: Practical Code Examples for Agentic Design Patterns

This appendix is an implementation companion to the SOP body. It preserves the SOP as the authoritative governance model while adding prototype-level code examples for common agentic design patterns. The examples are intended for learning, scaffolding, and repository examples/ or tests/ folders. They are not production-ready controls by themselves.

20.6.1 Third-Party Source, Code and License Notice

This appendix includes selected code examples that are either (1) adapted from code blocks in *Agentic Design Patterns* that include an explicit MIT License notice, or (2) newly authored for this whitepaper as short implementation skeletons inspired by the documented design patterns. Third-party notices are preserved where third-party code was adapted. Original appendix examples authored for this whitepaper are offered under the MIT License for use in the adopting repository, subject to the organization selecting and approving that license for its own distribution model.

20.6.2 MIT License Compliance Requirements

When MIT-licensed code is copied, adapted, redistributed, or included in an enterprise repository, preserve the copyright notice and permission notice in all copies or substantial portions of the software. Keep a license inventory for every example added to this SOP, the repository, or the generated implementation package.

MIT License permission notice for included code examples:

```
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

20.6.3 Pattern-to-SOP Crosswalk

Pattern	Primary SOP artifacts	Enterprise use
Prompt chaining	PLANS.md, skills/*/SKILL.md, schemas/plan.schema.json, evals/*	Use for multi-step decomposition with validation between steps.
Routing	agents/*/agent.yaml, tools/registry.yaml, escalation.md, policies/opa/tool_use.rego	Use for intent, skill, tool, model, or human-escalation selection.
Parallelization	orchestration runtime, schemas/handoff.schema.json, schemas/consensus.schema.json	Use only for independent analysis, retrieval, validation, or specialist review.
Reflection / producer-critic review	prompts/critic-prompts, evals/rubrics, ACCEPTANCE.md	Use to improve drafts and plans; critic output remains advisory until validated.
Tool use	tools/registry.yaml, schemas/tool-call.schema.json, policies/opa/tool_use.rego	Use only through approved wrappers with schemas, authz, logging, and policy checks.
Goal setting and monitoring	ACCEPTANCE.md, observability/slo-governance.md, evals/reports	Map local task goals to measurable evals, SLIs/SLOs, and approval triggers.
Resource-aware optimization	agent manifests, model profiles, runtime limits, observability metrics	Control cost and latency without bypassing quality, safety, policy, or data rules.

Evaluation and monitoring	evals/*, observability/*, audit/evidence-index.md	Use eval harnesses and telemetry to block promotion or reduce autonomy when needed.

20.6.4 Prompt Chaining Example

Purpose: Demonstrates a two-step extraction and transformation pipeline. This example is newly authored for this whitepaper and licensed under MIT for inclusion in adopting repositories.

SOP mapping: PLANS.md, skills/<name>/SKILL.md, schemas/plan.schema.json, evals/fixtures.

```
# MIT License
# Copyright (c) 2026 Garrett Spear
# Example authored for this whitepaper.

def prompt_chaining_example(llm, text: str) -> str:
    """Run extract -> transform as two auditable prompt-chain steps."""
    extract_prompt = (
        "Extract CPU, memory, and storage from this asset description. "
        "Return short bullet facts only.

" + text
    )
    extracted = llm(extract_prompt)

    transform_prompt = (
        "Convert the following facts into JSON with keys cpu, memory, storage. "
        "Return JSON only.

" + extracted
    )
    result_json = llm(transform_prompt)
    return result_json

# Production hardening: validate JSON schema, log both step IDs, and
# block downstream tool use if extraction confidence or schema validation fails.
```

20.6.5 Routing Example

Purpose: Demonstrates routing a request to a handler based on an LLM or classifier decision. Adapted in short form from an MIT-licensed routing example in Agentic Design Patterns.

```
# Copyright (c) 2025 Marco Fago
# https://www.linkedin.com/in/marco-fago/
# This code is licensed under the MIT License.
# Adapted for this whitepaper as a short-form routing skeleton.

def booking_handler(request: str) -> str:
    return f"Booking handler processed: {request}"

def info_handler(request: str) -> str:
    return f"Information handler processed: {request}"

def unclear_handler(request: str) -> str:
    return f"Clarification required for: {request}"
```

```
def route_request(request: str, classify) -> str:
    """classify(request) must return booker, info, or unclear."""
    decision = classify(request).strip().lower()
    routes = {
        "booker": booking_handler,
        "info": info_handler,
        "unclear": unclear_handler,
    }
    handler = routes.get(decision, unclear_handler)
    return handler(request)

# Production hardening: log route decisions, test false-route cases,
# add policy-denied and human-escalation routes, and restrict handlers by manifest.
```

20.6.6 Parallelization Example

Purpose: Demonstrates fan-out/fan-in execution for independent read-only analysis. This example is newly authored for this whitepaper and licensed under MIT for inclusion in adopting repositories.

```
# MIT License
# Copyright (c) 2026 Garrett Spear
# Example authored for this whitepaper.

import asyncio

async def summarize_source(fetch_and_summarize, source_id: str) -> dict:
    summary = await fetch_and_summarize(source_id)
    return {"source_id": source_id, "summary": summary}

async def parallel_research(fetch_and_summarize, source_ids: list[str]) -> list[dict]:
    tasks = [summarize_source(fetch_and_summarize, sid) for sid in source_ids]
    return await asyncio.gather(*tasks, return_exceptions=False)

# Production hardening: add timeouts, rate limits, idempotency keys,
# partial-failure handling, correlation IDs, and result aggregation schema.
```

20.6.7 Reflection / Producer-Critic Review Example

Purpose: Demonstrates producer-critic review for improving a plan or draft. This example is newly authored for this whitepaper and licensed under MIT for inclusion in adopting repositories.

```
# MIT License
# Copyright (c) 2026 Garrett Spear
# Example authored for this whitepaper.

def producer_critic_loop(llm, task: str, max_rounds: int = 2) -> str:
    draft = llm(f"Create a concise implementation plan for:
{task}")
    for _ in range(max_rounds):
        critique = llm(
            "You are a security and architecture critic. Identify only "
            "material gaps, unsafe assumptions, missing tests, or approval needs.

" + draft
        )
```

```
        if "no material gaps" in critique.lower():
            break
        draft = llm(
            "Revise the plan using this critique. Preserve safe constraints.

"
            f"Critique:
{critique}

Draft:
{draft}"
        )
    return draft

# Production hardening: cap iterations, store critique events, require
# deterministic tests/policy checks, and escalate unresolved high-risk findings.
```

20.6.8 Tool Use Example

Purpose: Demonstrates a minimal tool registry and governed tool dispatch pattern. This example is newly authored for this whitepaper and licensed under MIT for inclusion in adopting repositories.

```
# MIT License
# Copyright (c) 2026 Garrett Spear
# Example authored for this whitepaper.

TOOL_REGISTRY = {}

def register_tool(name: str, risk_tier: str):
    def wrapper(func):
        TOOL_REGISTRY[name] = {"func": func, "risk_tier": risk_tier}
        return func
    return wrapper

@register_tool("lookup_ticket", risk_tier="low")
def lookup_ticket(ticket_id: str) -> dict:
    return {"ticket_id": ticket_id, "status": "example", "summary": "demo"}

def call_tool(name: str, args: dict, authorize, audit):
    tool = TOOL_REGISTRY[name]
    authorize(tool_name=name, risk_tier=tool["risk_tier"], args=args)
    result = tool["func"](**args)
    audit(event_type="tool_call", tool=name, args=args, result=result)
    return result

# Production hardening: replace in-code registry with tools/registry.yaml,
# validate schemas, enforce scoped credentials, and log black-box event IDs.
```

20.6.9 Goal Setting and Monitoring Example

Purpose: Demonstrates iterative generation, review, and stop conditions based on explicit goals. Adapted in short form from an MIT-licensed goal-setting and monitoring example in Agentic Design Patterns.

```
# MIT License
# Copyright (c) 2025 Mahtab Syed
```

```
# https://www.linkedin.com/in/mahtabsyed/
# Adapted for this whitepaper as a short-form goal-monitoring skeleton.

def build_prompt(use_case: str, goals: list[str], previous: str = "", feedback: str = "") -> str:
    goal_text = "
".join(f"- {g}" for g in goals)
    return f"Use case:
{use_case}

Goals:
{goal_text}

Previous:
{previous}

Feedback:
{feedback}"

def generate_until_goals_met(llm, use_case: str, goals: list[str], max_iter: int = 5) -> str:
    code = ""
    feedback = ""
    for _ in range(max_iter):
        code = llm(build_prompt(use_case, goals, code, feedback))
        feedback = llm("Review this output against the goals. Return MET or specific gaps.

" + code)
        if feedback.strip().upper().startswith("MET"):
            break
    return code

# Production hardening: replace self-judgment-only checks with unit tests,
# rubric graders, human review for high-risk outputs, and eval thresholds.
```

20.6.10 Resource-Aware Optimization Example

Purpose: Demonstrates classifying work into lower-cost or higher-capability execution paths. Adapted in short form from an MIT-licensed resource-aware optimization example in Agentic Design Patterns.

```
# MIT License
# Copyright (c) 2025 Mahtab Syed
# https://www.linkedin.com/in/mahtabsyed/
# Adapted for this whitepaper as a short-form resource-routing skeleton.

def classify_prompt(prompt: str, classifier) -> str:
    category = classifier(prompt).strip().lower()
    if category not in {"simple", "reasoning", "internet_search"}:
        return "reasoning"
    return category

def choose_execution_path(prompt: str, classifier) -> dict:
    category = classify_prompt(prompt, classifier)
    if category == "simple":
        return {"model_profile": "quick", "tools": []}
```

```
    if category == "internet_search":
        return {"model_profile": "balanced", "tools": ["approved_search"]}
    return {"model_profile": "expert", "tools": []}

# Production hardening: enforce data rules before search, log model-routing
# rationale, monitor cost/latency, and deny optimization paths that reduce safety.
```

20.6.11 Evaluation and Monitoring Example

Purpose: Demonstrates an evaluation harness record that can become an evals/reports artifact. This example is newly authored for this whitepaper and licensed under MIT for inclusion in adopting repositories.

```
# MIT License
# Copyright (c) 2026 Garrett Spear
# Example authored for this whitepaper.

from dataclasses import dataclass, asdict

@dataclass
class EvalResult:
    case_id: str
    passed: bool
    score: float
    findings: list[str]
    latency_ms: int
    cost_usd: float

def evaluate_output(case_id: str, output: str, expected_terms: list[str], latency_ms: int, cost_usd:
float) -> dict:
    missing = [term for term in expected_terms if term.lower() not in output.lower()]
    score = 1.0 - (len(missing) / max(len(expected_terms), 1))
    result = EvalResult(
        case_id=case_id,
        passed=score >= 0.85,
        score=round(score, 2),
        findings=[f"missing:{m}" for m in missing],
        latency_ms=latency_ms,
        cost_usd=cost_usd,
    )
    return asdict(result)

# Production hardening: add security cases, policy-denied cases, prompt-injection
# cases, human override metrics, release thresholds, and black-box event links.
```

20.6.12 Production Hardening Checklist for Appendix F Examples

- Map the example to an agent manifest, skill, tool contract, schema, policy gate, eval, and observability event before operational use.
- Validate all model outputs before they are used as tool inputs, shell commands, SQL, Terraform, API payloads, tickets, or change records.
- Add deterministic tests, schema validation, policy checks, red-team cases, and failure-mode tests.
- Log route decisions, prompt-chain step IDs, critical; findings, tool calls, eval results, cost, latency, and human overrides to the black-box recorder.

- Use scoped credentials, rate limits, timeouts, idempotency keys, rollback plans, and kill switches for any write-capable or production-impacting pathway.
- Treat examples as prototype implementation aids, not as authorization to bypass the SOP body.